B.TECH FIRST YEAR | COMPUTER SCIENCE & ENGINEERING

PROGRAMMING FOR PROBLEM SOLVING

Control Structures in C Language

A Comprehensive Study Module

UNIT II

Subject Code	
Unit	Unit II – Control Structures
Level	
Language	C Programming (ANSI C)
Topics Covered	
Prepared By	Department of Computer Science

This module is prepared for academic use. All programs follow ANSI C standards.

TABLE OF CONTENTS

SECTION 01

Introduction to Control Structures

1.1 Definition

A **Control Structure** is a block of programming that determines the flow of execution of statements in a program. Control structures decide the order in which individual statements, instructions, or function calls are executed or evaluated. They are the backbone of any programming language.

1.2 Why Control Structures?

In the absence of control structures, a program executes each statement one after another in a strict top-to-bottom order. However, real-world problems demand decision making, repetition, and branching. Control structures provide this capability to the programmer.

1.3 Types of Control Structures in C

Type	Category	Statements in C	Purpose
Sequential	Default execution	Statement ; Statement;	Executes statements one by one
Conditional / Selection	Decision making	if, if-else, switch	Chooses among alternate paths
Iteration / Looping	Repetition	for, while, do-while	Repeats a block of statements
Jump / Transfer	Unconditional jump	break, continue, goto, return	Transfers control to a different location

1.4 Big Picture – Control Flow Overview

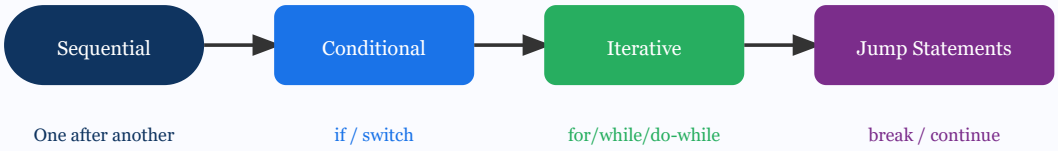

Fig 1.1 – Types of Control Structures in C

> 📌 **Exam Key Point:** Control structures allow programmers to control the flow of program execution without which all programs would only execute sequentially from top to bottom. The C language provides three broad categories: Sequential, Selection, and Iteration.

SECTION 02

Sequential Programming

2.1 Definition

Sequential Programming refers to the execution of program statements in the exact order they appear in the source code — from top to bottom, one at a time, without skipping or repeating any statement.

2.2 Explanation & Theory

In C, by default, execution is sequential. The CPU fetches each instruction from memory in order, executes it, and proceeds to the next instruction. There are no branches or loops involved. This is the simplest form of control flow and serves as the foundation for all other control structures.

A sequential program consists of: **declarations**, **input statements**, **computation statements**, and **output statements** — executed strictly in written order.

2.3 Step-by-Step Execution

1. Program starts at `main()` function.
2. Variable declarations are processed — memory is allocated.
3. Input statements (`scanf`) read data from the user.
4. Computation/assignment statements are executed in order.
5. Output statements (`printf`) display results.
6. Program reaches end of `main()` and terminates.

2.4 Sequential Execution Flow Diagram

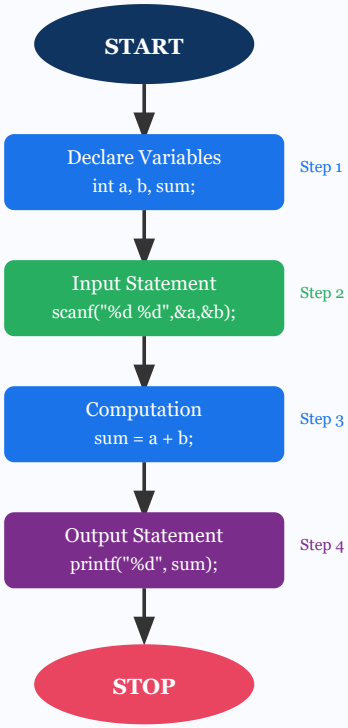

Fig 2.1 – Sequential Execution Flow Diagram

2.5 Sample Program – Sum of Two Numbers

```c
/* Sequential Program: Sum of Two Numbers */
#include <stdio.h>

int main() {
    int a, b, sum;            /* Step 1: Declare variables */

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);   /* Step 2: Input */

    sum = a + b;              /* Step 3: Computation */

    printf("Sum = %d\n", sum); /* Step 4: Output */

    return 0;
}
```

2.6 Advantages & Limitations

Advantages	Limitations

Simple and easy to understand	Cannot make decisions
Predictable execution order	Cannot repeat tasks (no loops)
Easy to debug	Not suitable for complex problems
Foundation of all programming	Limited to straight-line problems

SECTION 03

Decision Making in C

3.1 Overview of Decision Making

Decision making allows a program to evaluate a condition and choose different execution paths depending on whether the condition is **true** or **false**. In C, a condition evaluates to a non-zero value (true) or zero (false).

C provides the following decision-making constructs:

- **if statement** – One-way decision
- **if-else statement** – Two-way decision
- **Nested if / else-if ladder** – Multi-way decision
- **switch statement** – Multiple constant-based decision

3.2 Decision Structure Comparison Table

Construct	Decision Type	Condition Type	Best Use Case
if	One-way	Boolean / relational	Check a single condition
if-else	Two-way	Boolean / relational	True/false alternatives
else-if ladder	Multi-way	Range / complex	Multiple conditions/ranges
switch	Multi-way	Integer / char constant	Menu-driven programs

📘 **Note:** In C, any non-zero integer value is treated as **true**, and zero is treated as **false**. The relational operators (<, >, ==, !=, <=, >=) and logical operators (&&, ||, !) are commonly used to form conditions.

SECTION 04

The if Statement

4.1 Definition

The **if statement** is the simplest form of decision-making in C. It allows a block of code to be executed only when a specified condition evaluates to *true* (non-zero). If the condition is false, the block is entirely skipped.

4.2 Syntax

```
if (condition) { // body: statements executed only if condition is TRUE }
```

4.3 Step-by-Step Execution

1. The condition inside `if()` is evaluated.
2. If condition is **true** (non-zero) → execute the body statements.
3. If condition is **false** (zero) → skip the body entirely.
4. Control passes to the next statement after the if block.

4.4 Flowchart – if Statement

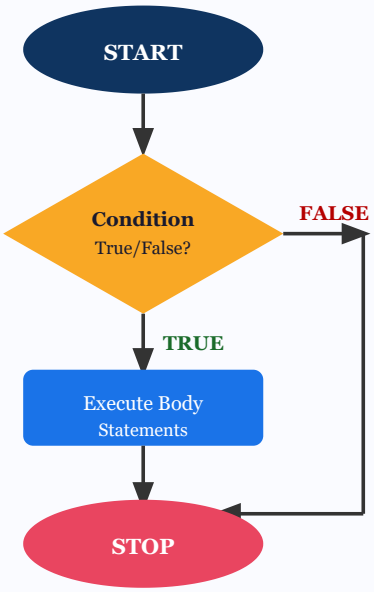

Fig 4.1 – Flowchart of if Statement

4.5 Sample Programs

```c
/* Example 1: Check if a number is positive */
#include <stdio.h>
int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n > 0) {
        printf("Number is positive.\n");
    }
    return 0;
}
```

```c
/* Example 2: Check even number */
#include <stdio.h>
int main() {
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);
    if (num % 2 == 0) {
        printf("%d is Even.\n", num);
    }
    printf("Program ends.\n");
    return 0;
}
```

4.6 Advantages & Limitations

Advantages	Limitations
Simple and readable	Only handles the true case
Skips unnecessary code	No alternative path for false
Can test complex conditions	Multiple conditions need nesting

4.7 Applications

- Checking eligibility criteria (age >= 18 to vote)
- Validating user input
- Checking divisibility

- Access control systems

SECTION 05

if-else Statement & Nested if

5.1 if-else Statement – Definition

The **if-else** statement provides a two-way decision. When the condition is true, one block executes; when false, an alternative block executes. This ensures that exactly one of the two blocks always runs.

5.2 Syntax – if-else

```
if (condition) { // Block A: executed when condition is TRUE } else
{ // Block B: executed when condition is FALSE }
```

5.3 Flowchart – if-else Statement

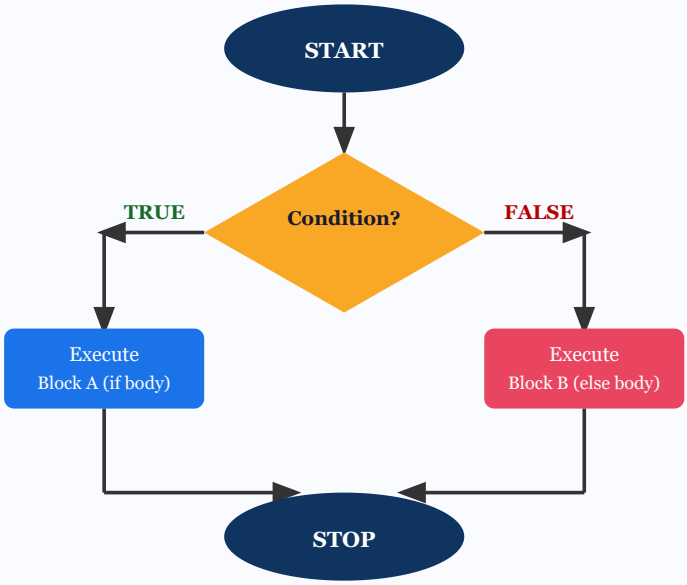

Fig 5.1 – Flowchart of if-else Statement

5.4 Sample Program – if-else

```c
/* Find greater of two numbers using if-else */
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter two numbers: ");
```

```
    scanf("%d %d", &a, &b);
    if (a > b) {
        printf("%d is greater.\n", a);
    } else {
        printf("%d is greater.\n", b);
    }
    return 0;
}
```

5.5 Nested if – Definition

A **Nested if** is an if statement placed inside another if or else block. It is used when a decision requires multiple levels of evaluation. Also called the *else-if ladder* when chained sequentially.

5.6 Syntax – Nested if / else-if Ladder

```
if (condition1) { // Block 1 } else if (condition2) { // Block 2 }
else if (condition3) { // Block 3 } else { // Default Block }
```

5.7 Nested if Decision Tree Diagram

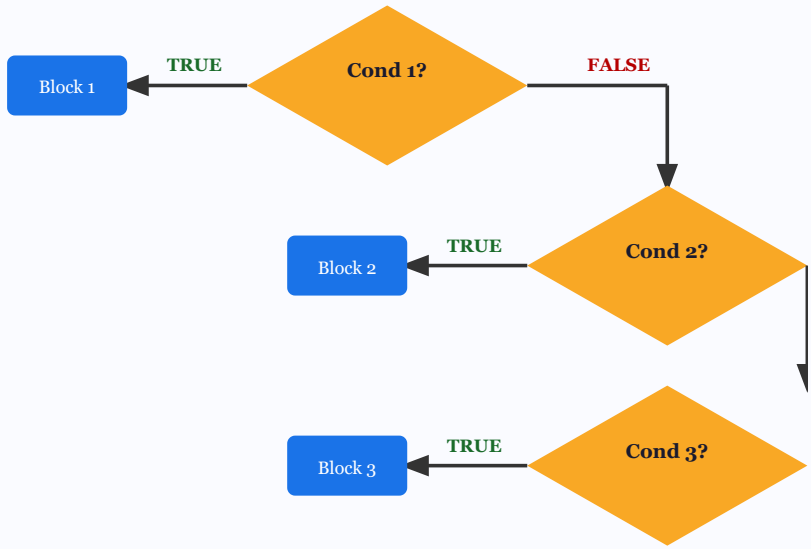

Fig 5.2 – Nested if Decision Tree

5.8 Sample Program – Grade Classification (else-if ladder)

```
/* Grade Classification using else-if ladder */
#include <stdio.h>
int main() {
    int marks;
```

```
    printf("Enter marks (0-100): ");
    scanf("%d", &marks);
    if (marks >= 90) {
        printf("Grade: O (Outstanding)\n");
    } else if (marks >= 80) {
        printf("Grade: A (Excellent)\n");
    } else if (marks >= 70) {
        printf("Grade: B (Good)\n");
    } else if (marks >= 60) {
        printf("Grade: C (Average)\n");
    } else {
        printf("Grade: F (Fail)\n");
    }
    return 0;
}
```

SECTION 06

switch Statement

6.1 Definition

The **switch** statement is a multi-way decision structure that tests the value of an expression against a list of integer or character constants (cases). The matching case's statements are executed. A `default` case handles unmatched values.

6.2 Syntax

```
switch (expression) { case constant1: // statements break; case constant2: // statements break; ... default: // statements (optional) }
```

6.3 Step-by-Step Execution

1. Evaluate the `switch(expression)`.
2. Compare the value with each `case` constant top to bottom.
3. When a match is found, execute all statements from that case onward.
4. `break` exits the switch block.
5. If no case matches and `default` exists, execute the default block.
6. Without `break`, *fall-through* occurs (next case executes too).

6.4 switch Case Flow Diagram

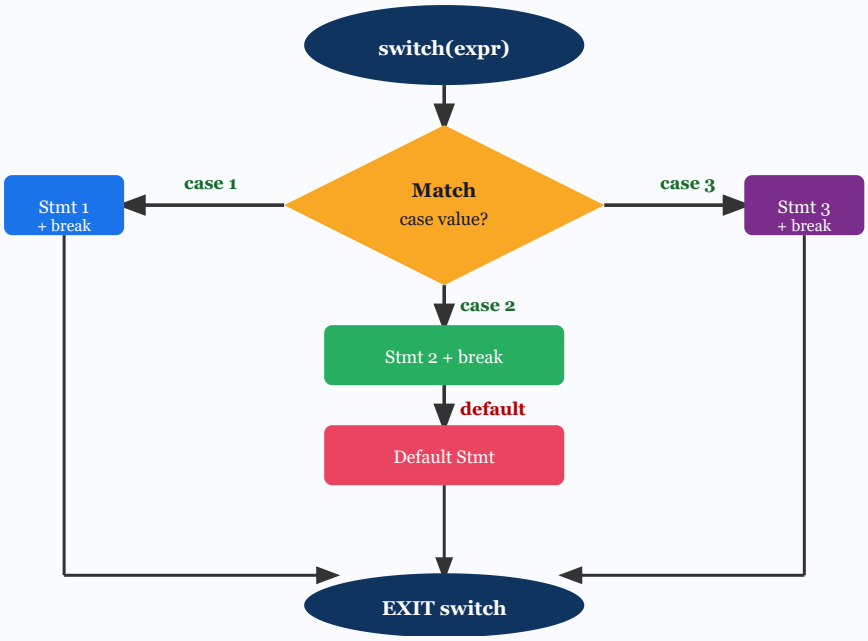

Fig 6.1 – switch Statement Flow Diagram

6.5 Sample Program – Calculator using switch

```c
/* Simple Calculator using switch statement */
#include <stdio.h>
int main() {
    float a, b, result;
    char op;
    printf("Enter: num op num (e.g., 5 + 3): ");
    scanf("%f %c %f", &a, &op, &b);
    switch (op) {
        case '+': result = a + b; printf("%.2f\n", result); break;
        case '-': result = a - b; printf("%.2f\n", result); break;
        case '*': result = a * b; printf("%.2f\n", result); break;
        case '/':
            if (b != 0) { result = a / b; printf("%.2f\n", result); }
            else        { printf("Division by zero!\n"); }
            break;
        default:
            printf("Invalid operator!\n");
    }
    return 0;
}
```

6.6 if vs switch Comparison Table

Feature	if / else-if	switch

Condition type	Any Boolean / relational	Integer / char constants only
Range testing	Yes (e.g., x > 10 && x < 20)	No (exact values only)
Fall-through	Not applicable	Yes (without break)
Float conditions	Yes	No
Readability for menus	Less readable	Very readable
Speed (many branches)	Slower (sequential check)	Faster (jump table)
Default case	else block	default: block

⚠️ **Important:** Without a `break` statement in each case, **fall-through** occurs — execution continues into the next case even if it doesn't match. This is sometimes intentional but is often a bug. Always use `break` unless fall-through is desired.

SECTION 07

Looping Concepts

7.1 Definition of a Loop

A **loop** is a control structure that repeatedly executes a block of statements as long as a specified condition remains true. When the condition becomes false, the loop terminates and control passes to the next statement after the loop.

7.2 Parts of a Loop

Part	Description	Example
Initialization	Set the loop control variable to a starting value	`i = 0`
Condition	Test whether the loop should continue	`i < 10`
Body	Statements executed in each iteration	`printf("%d", i);`
Update	Modify the loop control variable	`i++`

7.3 Types of Loops in C

- **for loop** – When the number of iterations is known in advance.
- **while loop** – When the number of iterations is unknown; condition tested before body.
- **do-while loop** – Similar to while, but body executes at least once; condition tested after body.

7.4 Loop Comparison Table

Feature	for Loop	while Loop	do-while Loop
Condition check	Before body (entry-controlled)	Before body (entry-controlled)	After body (exit-controlled)
Minimum executions	0 (if false initially)	0 (if false initially)	1 (always executes once)

Initialization	Inside for header	Before loop	Before loop
Update	Inside for header	Inside body	Inside body
Best used when	Count known	Count unknown	Menu, user input
Semicolon at end	No	No	Yes (after while condition)

> 📘 **Entry-controlled vs Exit-controlled:** In **entry-controlled** loops (for, while), the condition is tested *before* executing the body. In **exit-controlled** loops (do-while), the body executes first, then the condition is checked.

SECTION 08

The for Loop

8.1 Definition

The **for loop** is an entry-controlled loop used when the number of iterations is known beforehand. It combines initialization, condition check, and update expression in a single line, making it compact and readable.

8.2 Syntax

```
for (initialization; condition; update) { // body statements }
```

8.3 Step-by-Step Execution

1. **Initialization**: Execute once at the start (e.g.,`i = 0`).
2. **Condition check**: If true → go to step 3; if false → exit loop.
3. **Execute body**: Run all statements inside { }.
4. **Update**: Execute update expression (e.g.,`i++`).
5. Go back to Step 2 and repeat.

8.4 for Loop Flowchart

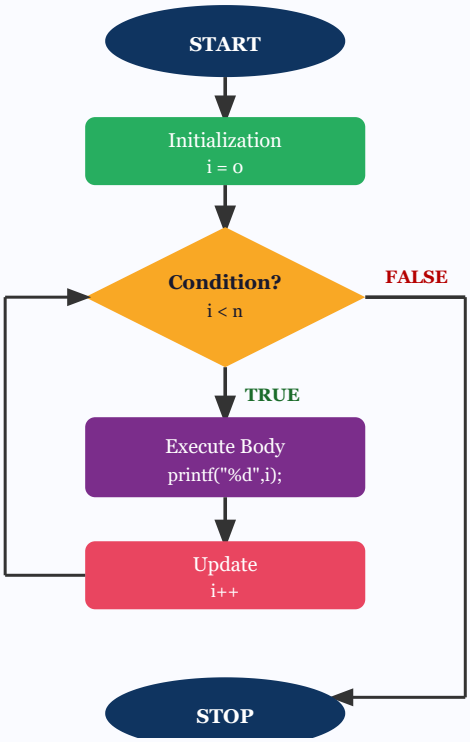

Fig 8.1 – for Loop Flowchart

8.5 Sample Programs

```c
/* Print 1 to 10 using for loop */
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i <= 10; i++) {
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
/* Output: 1 2 3 4 5 6 7 8 9 10 */
```

```c
/* Sum of first N natural numbers */
#include <stdio.h>
int main() {
    int i, n, sum = 0;
    printf("Enter N: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        sum += i;
    }
    printf("Sum = %d\n", sum);
```

```
    return 0;
}
```

8.6 Loop Execution Trace Table (i=1 to 5)

Iteration	i value	Condition (i≤5)	Output	After i++
1st	1	TRUE	1	i=2
2nd	2	TRUE	2	i=3
3rd	3	TRUE	3	i=4
4th	4	TRUE	4	i=5
5th	5	TRUE	5	i=6
6th	6	FALSE	–	Exit loop

SECTION 09

The while Loop

9.1 Definition

The **while loop** is an entry-controlled loop. The condition is tested *before* the loop body executes. If the condition is false from the start, the body never executes. It is used when the number of iterations is not known in advance.

9.2 Syntax

```
initialization; while (condition) { // body statements update; }
```

9.3 Step-by-Step Execution

1. Initialize the loop control variable before the while statement.
2. Evaluate the condition. If false → exit loop. If true → step 3.
3. Execute the body statements.
4. Update the loop control variable inside the body.
5. Return to step 2.

9.4 while Loop Flowchart

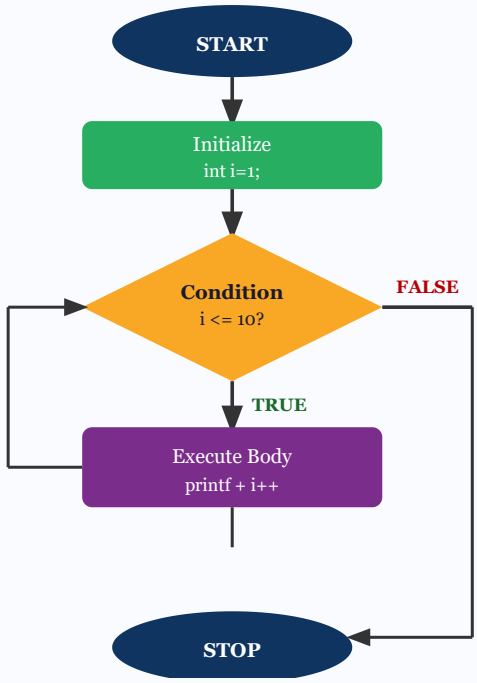

Fig 9.1 – while Loop Flowchart

9.5 Sample Programs

```c
/* Print digits while user enters positive numbers */
#include <stdio.h>
int main() {
    int num;
    printf("Enter numbers (-1 to stop): \n");
    scanf("%d", &num);
    while (num != -1) {
        printf("You entered: %d\n", num);
        scanf("%d", &num);
    }
    printf("Loop ended.\n");
    return 0;
}
```

```c
/* Factorial using while loop */
#include <stdio.h>
int main() {
    int n, i = 1;
    long fact = 1;
    printf("Enter n: ");
    scanf("%d", &n);
    while (i <= n) {
        fact *= i;
        i++;
    }
```

```
    printf("%d! = %ld\n", n, fact);
    return 0;
}
```

SECTION 10

The do-while Loop

10.1 Definition

The **do-while loop** is an exit-controlled loop. Unlike for and while, the body executes *at least once* regardless of the condition. The condition is checked *after* the body executes.

10.2 Syntax

```
initialization; do { // body statements update; } while
(condition); ← NOTE: semicolon is mandatory here
```

10.3 do-while Loop Flowchart

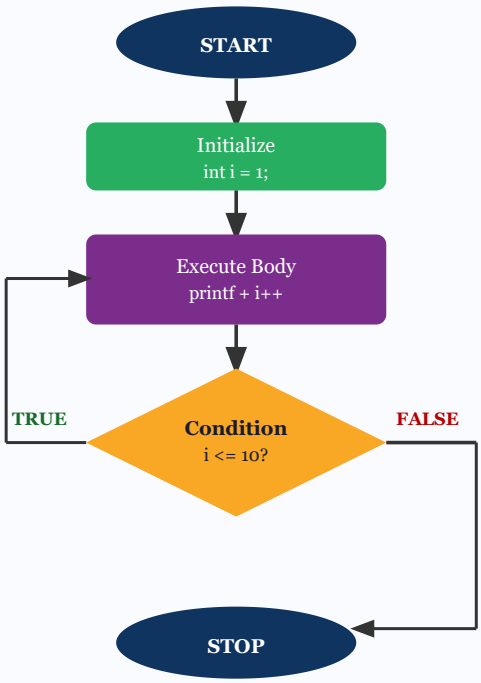

Fig 10.1 – do-while Loop Flowchart

10.4 Sample Programs

```c
/* Menu-driven program using do-while */
#include <stdio.h>
int main() {
    int choice;
```

```c
    do {
        printf("\n--- MENU ---\n");
        printf("1. Hello\n2. World\n3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1: printf("Hello!\n"); break;
            case 2: printf("World!\n"); break;
            case 3: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 3);
    return 0;
}
```

📌 **Key Difference:** The do-while loop is the only loop in C where the body is **guaranteed to execute at least once** regardless of the condition. This makes it ideal for menu-driven programs and input validation where you always want to show the prompt at least once.

SECTION 11

break Statement

11.1 Definition

The **break** statement is a jump statement used to immediately terminate the execution of the nearest enclosing loop (`for`, `while`, `do-while`) or `switch` statement. After `break`, control transfers to the statement immediately following the loop/switch.

11.2 Syntax

```
break;
```

11.3 break Statement Flow Diagram

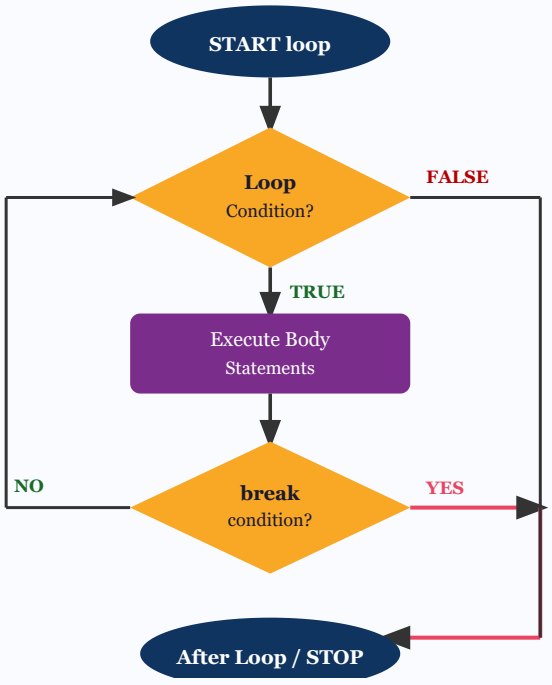

Fig 11.1 – break Statement Flow (red arrow = break path)

11.4 Sample Program

```c
/* Find first multiple of 7 in range 1-50 */
#include <stdio.h>
int main() {
    int i;
```

```c
    for (i = 1; i <= 50; i++) {
        if (i % 7 == 0) {
            printf("First multiple of 7: %d\n", i);
            break;  /* exits the for loop immediately */
        }
    }
    printf("After the loop.\n");
    return 0;
}
/* Output: First multiple of 7: 7 */
```

11.5 Applications of break

- Exiting a loop when a search target is found
- Stopping infinite loops on a specific condition
- Preventing fall-through in switch statements
- Early termination in error handling

SECTION 12

continue Statement

12.1 Definition

The **continue** statement is a jump statement that skips the remaining statements of the current iteration and immediately transfers control to the update expression (in `for`) or the condition check (in `while`/`do-while`) to start the next iteration. Unlike `break`, it does not exit the loop entirely.

12.2 Syntax

```
continue;
```

12.3 continue Statement Flow Diagram

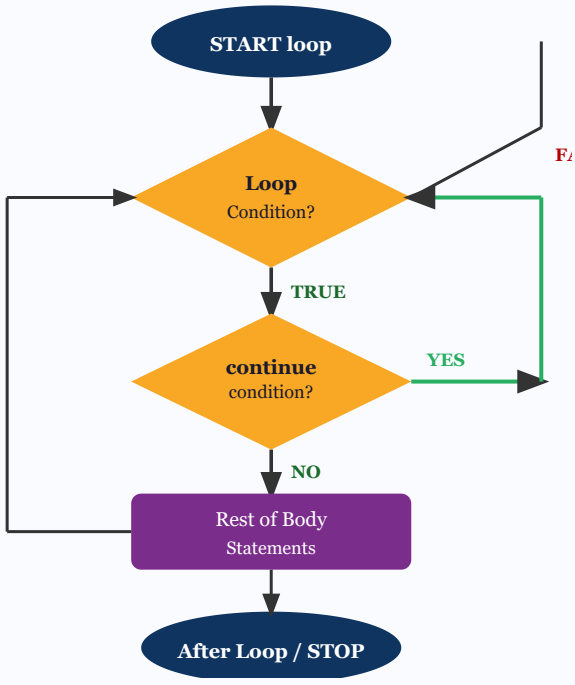

Fig 12.1 – continue Statement Flow (green arrow = skip to next iteration)

12.4 Sample Program

```
/* Print all numbers from 1-10 except multiples of 3 */
#include <stdio.h>
```

```c
int main() {
    int i;
    for (i = 1; i <= 10; i++) {
        if (i % 3 == 0) {
            continue;  /* skip 3, 6, 9 */
        }
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
/* Output: 1 2 4 5 7 8 10 */
```

12.5 break vs continue Comparison

Feature	break	continue
Effect on loop	Exits loop completely	Skips current iteration only
Loop continues?	No	Yes
Used in switch?	Yes	No (not meaningful)
Control goes to	Statement after loop/switch	Next iteration (update/condition)
Use case	Early exit, search done	Skip specific iterations

SECTION 13

Questions & Model Answers

Part A — 2-Mark Questions

Q1. What is a control structure in C? 2 Marks

A **control structure** is a programming construct that determines the order of execution of statements in a C program. It controls how the program flows between different instructions. C provides three types of control structures:

(1) **Sequential** – Statements execute one after another in written order. (2) **Selection/Conditional** – Executes specific statements based on conditions (if, switch). (3) **Iterative/Loop** – Repeats a block of statements multiple times (for, while, do-while).

```c
/* Example: all three in one program */
int x = 5;                    /* sequential */
if (x > 0) printf("+");      /* conditional */
for(int i=0; i<3; i++) {}   /* iterative */
```

Q2. What is the difference between entry-controlled and exit-controlled loops? 2 Marks

In an **entry-controlled loop**, the condition is tested *before* the loop body executes. If the condition is false initially, the body is never executed. Examples: `for` and `while` loops.

In an **exit-controlled loop**, the loop body executes first and the condition is tested *after*. The body always executes at least once, even if the condition is false. Example: `do-while` loop.

```c
int x = 10;
while (x < 5) { printf("never"); } /* 0 times */
do { printf("once"); } while (x < 5); /* 1 time */
```

Q3. What is a switch statement? When is it preferred over if-else? 2 Marks

The **switch statement** is a multi-way branching construct that evaluates an integer or character expression and transfers control to the matching `case` label. It is preferred over if-else when there are many discrete constant values to compare, as it improves readability and can be more efficient (jump table).

switch is preferred for: menu selection, weekday names, grade mapping, operator selection. It cannot test ranges or floating-point values.

```
switch(day) {
    case 1: printf("Monday");   break;
    case 2: printf("Tuesday");  break;
    default:printf("Other");
}
```

Q4. What is the purpose of the break statement in a loop? 2 Marks

The **break** statement immediately terminates the nearest enclosing loop or switch statement when encountered. After executing break, control jumps to the statement following the loop/switch, bypassing all remaining iterations. It is used for early termination when a desired result is found or an error occurs.

Use cases include: stopping a search loop once the target is found, handling errors inside loops, and preventing fall-through in switch blocks.

```
for(int i=1; i<=100; i++) {
    if(i == 5) break; /* stops at 5 */
    printf("%d ", i);  /* prints 1 2 3 4 */
}
```

Part B — 5-Mark Questions

Q5. Explain the if-else statement with syntax, flowchart description, and example program. 5 Marks

The **if-else statement** is a two-way decision-making construct in C. It tests a condition and executes one of two blocks: the *if body* when the condition is true, or the *else body* when the condition is false. Exactly one block always executes.

Syntax:

```
if (condition) { // statements when true } else { //
statements when false }
```

Execution steps: (1) Evaluate the condition in parentheses. (2) If true (non-zero): execute the if block. (3) If false (zero): execute the else block. (4) Continue after the entire if-else.

Flowchart: A diamond decision box with two paths — TRUE goes to Block A, FALSE goes to Block B. Both paths merge at a single endpoint below.

Advantages: Guarantees one block always executes; handles both true and false cases cleanly. **Limitations:** Only two paths; for multiple paths, else-if or switch is needed.

```c
/* Check if a student passes or fails */
#include <stdio.h>
int main() {
    int marks;
    printf("Enter marks: ");
    scanf("%d", &marks);
    if (marks >= 50) {
        printf("Result: PASS\n");
    } else {
        printf("Result: FAIL\n");
    }
    return 0;
}
/* Input: 72  → Output: PASS
   Input: 35  → Output: FAIL */
```

Q6. Explain the difference between for loop and while loop. Give example programs for each. 5 Marks

Both **for** and **while** are entry-controlled loops in C. However, they differ in structure and typical use cases:

Aspect	for Loop	while Loop
Structure	Init, condition, update in header	Init before; update inside body
Best used when	Number of iterations known	Number of iterations unknown
Readability	More compact	More flexible
Execution	Init → Cond → Body → Update	Init → Cond → Body (with update)

In the `for` loop, all loop-control elements (initialization, condition, update) are in one line, making it easier to understand count-based loops. The `while` loop is more natural when reading from input until a sentinel value.

```c
/* for loop: print squares of 1 to 5 */
#include <stdio.h>
int main() {
    for (int i = 1; i <= 5; i++) {
        printf("%d^2 = %d\n", i, i*i);
    }
    return 0;
}

/* while loop: sum until 0 is entered */
int main() {
    int n, sum = 0;
    scanf("%d", &n);
    while (n != 0) {
        sum += n;
        scanf("%d", &n);
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

Q7. Explain the switch statement with syntax and a complete example. What is fall-through? 5 Marks

The **switch statement** evaluates an integer or character expression and branches to the matching `case` label. If no match is found, the optional `default` block executes.

Syntax:

```
switch(expression) { case constant1: statements; break; case constant2: statements; break; default: statements; }
```

Rules: The switch expression must yield an integer or character. Case values must be constants (not variables or ranges). The `break` statement is needed to exit after each case. Without `break`, execution "falls through" to the next case automatically.

Fall-through: When `break` is omitted, C continues executing all subsequent case blocks until a `break` or end of switch is reached. This can be intentional (grouping cases) or unintentional (bug).

```c
/* Day of week using switch (with intentional fall-through) */
#include <stdio.h>
int main() {
    int day;
    printf("Enter day (1-7): ");
    scanf("%d", &day);
    switch (day) {
        case 1: printf("Monday\n");    break;
        case 2: printf("Tuesday\n");   break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n");  break;
        case 5: printf("Friday\n");    break;
        case 6:
        case 7: printf("Weekend!\n");  break; /* fall-through: 6 and
        default: printf("Invalid!\n");
    }
    return 0;
}
```

Q8. Write a program using loops to print the multiplication table of a given number. Explain the loop used. 5 Marks

The program reads a number from the user and prints its multiplication table from 1 to 10. A **for loop** is ideal here because the number of iterations (1 to 10) is fixed and known in advance. The loop iterates with the multiplier starting from 1 and increasing by 1 each time until it reaches 10.

Loop analysis: Initialization: `i=1`; Condition: `i<=10`; Update: `i++`; Body: prints the product.

```c
/* Multiplication Table using for loop */
#include <stdio.h>
int main() {
    int n, i;
    printf("Enter a number: ");
    scanf("%d", &n);
    printf("\n--- Multiplication Table of %d ---\n", n);
    for (i = 1; i <= 10; i++) {
        printf("%d x %2d = %3d\n", n, i, n*i);
    }
    return 0;
}

/* Sample Output for n=5:
   5 x  1 =   5
   5 x  2 =  10
   5 x  3 =  15
   ...
   5 x 10 =  50 */
```

Execution Table for n=5 (first 3 rows):

i	Condition (i≤10)	n*i	Output
1	TRUE	5	5 x 1 = 5
2	TRUE	10	5 x 2 = 10
10	TRUE	50	5 x 10 = 50
11	FALSE	–	Exit

Q9. Explain break and continue statements with examples. How do they differ? 5 Marks

Both **break** and **continue** are jump statements used inside loops and switch. They modify the normal sequential flow but in different ways.

break statement: Immediately exits the entire loop or switch. Control moves to the statement after the loop. Used when we want to stop execution early upon meeting a condition.

continue statement: Skips only the remaining statements of the *current iteration* and jumps to the next iteration. The loop continues running. Used when we want to skip specific values but continue the loop.

Key Difference: break = exit loop entirely; continue = skip current iteration only.

```c
/* break: stop when number is found */
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i <= 10; i++) {
        if (i == 6) break;
        printf("%d ", i);
    }
    /* Output: 1 2 3 4 5  (stops at 6) */

    printf("\n");

    /* continue: skip even numbers */
    for (i = 1; i <= 10; i++) {
        if (i % 2 == 0) continue;
        printf("%d ", i);
    }
    /* Output: 1 3 5 7 9  (skips even numbers) */
    return 0;
}
```

In the break example, the loop exits at i=6. In the continue example, the loop runs all 10 iterations but skips printing when i is even.

Part C — 10-Mark Question

Q10. Write a complete C program to implement a student result system using all control structures (if-else, switch, for loop, break, continue). Explain each part. 10 Marks

The following program demonstrates a comprehensive student result system. It reads marks for multiple subjects, computes the average, assigns a grade using switch, and uses loop control statements for navigation.

```c
/* Complete Student Result System */
#include <stdio.h>

int main() {
    int n, i, marks[10], sum = 0, valid = 0;
    float avg;
    char grade;
    int choice;

    printf("Enter number of subjects (1-10): ");
    scanf("%d", &n);

    /* for loop: read marks for each subject */
    for (i = 0; i < n; i++) {
        printf("Enter marks for Subject %d (0-100): ", i+1);
        scanf("%d", &marks[i]);

        /* if: validate marks */
        if (marks[i] < 0 || marks[i] > 100) {
            printf("Invalid marks! Skipping subject %d.\n", i+1);
            continue;  /* skip invalid entry */
        }

        /* if: check if below threshold */
        if (marks[i] < 35) {
            printf("Failed in subject %d. Stopping further input.\n"
            sum += marks[i];
            valid++;
            break;   /* break: stop input on fail */
        }

        sum += marks[i];
        valid++;
    }

    /* if-else: compute average */
    if (valid > 0) {
        avg = (float)sum / valid;
```

```
        printf("\nAverage Marks: %.2f\n", avg);
    } else {
        printf("No valid subjects entered.\n");
        return 1;
    }

    /* else-if ladder: assign grade code */
    int gradeCode;
    if      (avg >= 90) gradeCode = 1;
    else if (avg >= 75) gradeCode = 2;
    else if (avg >= 60) gradeCode = 3;
    else if (avg >= 50) gradeCode = 4;
    else                gradeCode = 5;

    /* switch: print grade description */
    switch (gradeCode) {
        case 1: printf("Grade: O  - Outstanding\n");  break;
        case 2: printf("Grade: A  - Excellent\n");    break;
        case 3: printf("Grade: B  - Good\n");         break;
        case 4: printf("Grade: C  - Satisfactory\n"); break;
        case 5: printf("Grade: F  - FAIL\n");         break;
        default:printf("Error in grade computation!\n");
    }

    return 0;
}
```

Explanation of control structures used:

- **for loop**: Iterates through each subject to read marks.
- **continue**: Skips subjects with invalid marks (<0 or >100).
- **break**: Stops input early if a student fails a subject.
- **if-else**: Checks if valid subjects were entered before computing average.
- **else-if ladder**: Assigns a numeric grade code based on average.
- **switch**: Prints the grade description based on the numeric code.

SECTION 14

Quick Revision Sheet

14.1 Syntax Summary of All Control Structures

if Statement

```
if (cond) { stmts; }
```

if-else Statement

```
if (cond) { stmts; } else { stmts; }
```

else-if Ladder

```
if (c1) { } else if (c2) { } else if (c3) { } else { }
```

switch Statement

```
switch (expr) { case c1: ...; break; case c2: ...; break; default: ...; }
```

for Loop

```
for (init; cond; upd) { body; }
```

while Loop

```
init; while (cond) { body; update; }
```

do-while Loop

```
init; do { body; update; } while (cond);
```

break & continue

```
break; // exit loop continue; // skip iteration
```

14.2 Loop Differences – At a Glance

Property	for	while	do-while
Control type	Entry	Entry	Exit
Min executions	0	0	1
Init location	Header	Before loop	Before loop
Update location	Header	Inside body	Inside body
Semicolon at end	No	No	Yes (while(cond);)

Best for	Fixed count	Unknown count	At-least-once tasks

14.3 Key Points for Exam

- **if** executes body only when condition is **true**. No else means nothing happens on false.
- **switch** works with **integer and char constants only** — NOT float, NOT ranges.
- Missing **break** in switch causes **fall-through** (executes next case too).
- **do-while** always executes body **at least once** — the only exit-controlled loop in C.
- **break** exits the **innermost** loop or switch only, not outer loops.
- **continue** skips to the **next iteration**, does not exit the loop.
- Infinite loop: `for(;;)` or `while(1)` — needs break to terminate.
- Nested loops: break only exits the **innermost** loop.
- In **for loop**, any of the three parts (init, cond, update) can be omitted but semicolons must remain.
- Loop control variable should always be updated to avoid **infinite loops**.

14.4 Flowchart Quick Summary

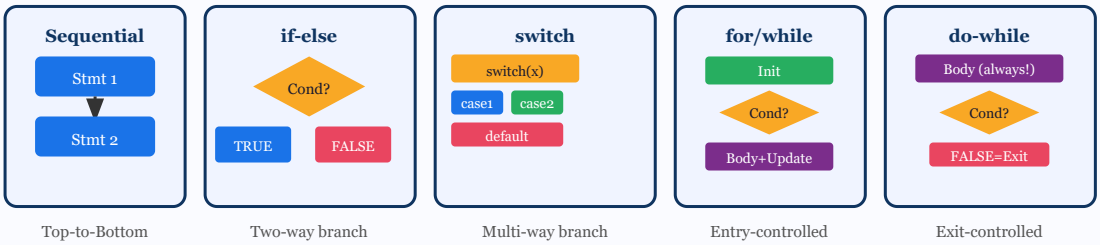

Fig 14.1 – Visual Summary of All Control Structures

14.5 Common Errors to Avoid

Mistake	Consequence	Fix
Using = instead of == in condition	Assignment, not comparison (always true)	Use == for equality check
Missing break in switch	Fall-through to next case	Add break after each case
Not updating loop variable	Infinite loop	Always include i++ or similar

Missing ; after do-while condition	Syntax error	while(cond); ← required
Float in switch expression	Compilation error	Use integer or char only
Off-by-one in loop bounds	Wrong iterations	Trace carefully: i<n vs i<=n

REFERENCES

Textbook & Reference Books

Prescribed Textbooks

[T1] Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd Edition, Prentice Hall, 1988. — The authoritative reference for the C language, written by its creators. Covers all control structures with precise definitions and idiomatic usage.

[T2] Byron S. Gottfried, *Schaum's Outline of Programming with C*, McGraw-Hill, 1996. — Comprehensive problem-solving approach with hundreds of solved examples covering loops, conditionals, and all control structures.

Reference Books

[R1] E. Balagurusamy, *Computing Fundamentals and C Programming*, McGraw-Hill Education India, 2008. — A widely used textbook for Indian university curricula. Covers all Unit II topics with detailed explanations suitable for first-year B.Tech students.

[R2] Rema Theraja, *Programming in C*, Oxford University Press, 2016. — Provides structured explanations of control structures with flowcharts, examples, and university-level exercises.

[R3] Behrouz A. Forouzan, Richard F. Gilberg, and Mala Gupta Prasad, *C Programming: A Problem Solving Approach*, 3rd Edition, Cengage Learning, latest edition. — Problem-centric approach ideal for engineering students, covering sequential, conditional, and iterative constructs with real-world applications.

Online Resources

[O1] **cprogramming.com** – Tutorials and practice problems for C language control structures.

[O2] **tutorialspoint.com/cprogramming** – Comprehensive reference with live code examples.

[O3] **NPTEL – Programming in C (IIT lectures)** – Video lectures by IIT faculty covering all Unit II topics systematically.

📘 END OF UNIT II – CONTROL STRUCTURES

Subject: Programming for Problem Solving (C Language) | B.Tech 1st Year

All programs conform to ANSI C standard. Compiled with GCC / Turbo C.